```
        response = self.client.get(reverse('authors'))
        self.assertEqual(response.status_code, 200)
        self.assertTrue('is_paginated' in response.context)
        self.assertTrue(response.context['is_paginated'] == True)
        self.assertTrue(len(response.context['author_list']) == 10)

    def test_lists_all_authors(self):
        # Get second page and confirm it has (exactly) remaining 3 items
        response = self.client.get(reverse('authors')+'?page=2')
        self.assertEqual(response.status_code, 200)
        self.assertTrue('is_paginated' in response.context)
        self.assertTrue(response.context['is_paginated'] == True)
        self.assertTrue(len(response.context['author_list']) == 3)
```

Templates

Templates can be checked through the test APIs provided by Django. It helps you discover whether proper views are called by the right template, and enable you to verify all the details being sent to the user.

In conclusion, Django tools can help you create your unit and integration tests. Here, we have tried to provide the most basic of testing Django web application. There are numerous Django clients that you can utilise for simulating the tests and achieve greater automation. Knowing how to create custom tests can help your career and enable you to make more robust web applications. So, don't just sit back; think about how you can ace Django web app testing!